

In one line: A SwiftUI app is a value: `@main struct MyApp: App` whose body returns **scenes**. A scene is a **template** the system instantiates into one or more **windows** (`UIWindowScenes`), each with its **own** view tree, `@State`, `@SceneStorage` and `scenePhase`. UIKit's delegates come back through an **adaptor**; URLs, activities and background wake-ups are delivered to scenes by **modifiers**.

How it works

- **@main** on an App generates `main()`. `App.init()` runs once, before any scene. `@State` on the App = process-wide model, injected into every window via `.environment(store)`; state in a root view = one copy **per window**.
- **scenePhase** (`.active` / `.inactive` / `.background`): in a view = the *containing* scene; in the App = **aggregate** — `.active` if any scene is, `.background` = app going to background. `.inactive` = visible, not interactive (switcher, Control Center) — pause, don't tear down.
- **UIKit hooks:** `@UIApplicationDelegateAdaptor` — once, in the App (twice = runtime error); SwiftUI creates it; an `ObservableObject` one lands in the environment. Push token, SDK setup. Scene delegate: `delegateClass` on the `UISceneConfiguration` returned from `configurationForConnecting`.
- **Incoming:** `.onOpenURL` — custom schemes *and* Universal Links, as URLs; `.onContinueUserActivity(type)` — Handoff, Spotlight; `.handlesExternalEvents(matching:)` picks the window.
- **Outgoing** (environment): `openURL` (own `OpenURLAction`: `.handled` / `.discarded` / `.systemAction`); `openWindow(id:/value:)` (16) — a window already showing the value comes forward; `dismissWindow` (17); `supportsMultipleWindows`.
- **Persistence:** `@SceneStorage` — per window, system-restored, small, **no secrets**, dies with the scene. `@AppStorage` — `UserDefaults`, shared by all windows.
- **.backgroundTask(.appRefresh("id"))** (16): done when the closure **returns**, then suspended; cancelled on timeout. Still needs the id in `BGTaskSchedulerPermittedIdentifiers` + a *submitted* `BGAppRefreshTaskRequest`.
- **.commands { CommandMenu ... }**: macOS menu bar; iPadOS: Cmd-hold HUD + menu bar.

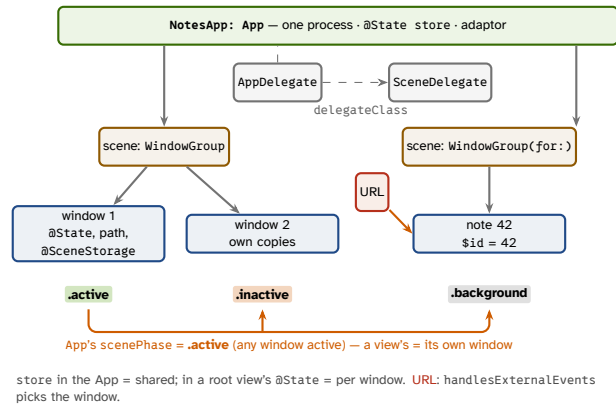
Scene types

<code>WindowGroup</code>	all	template; many windows on iPad / Mac
<code>WindowGroup(for: T.self)</code>	16	a window per value
<code>DocumentGroup</code>	14	browser, open / save, window per doc
<code>Window</code>	macOS 13	single unique window, not iOS
<code>Settings, MenuBarExtra</code>	macOS	preferences; menu-bar item

Remember

"App = the process, Scene = a template, Window = an instance with its own state and phase."

App, scenes, windows, phases



Example

```
@main struct NotesApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) var delegate
    @State private var store = NoteStore() // shared
    @Environment(\.scenePhase) private var phase // aggregate
    var body: some Scene {
        WindowGroup { RootView() }.environment(store)
        .onChange(of: phase) { _, p in
            if p == .background { store.save(); scheduleRefresh() }
            .backgroundTask(.appRefresh("com.example.sync")) {
                await store.sync() // return = done
            }
        }
        WindowGroup("Note", id: "note", for: Note.ID.self) { $id in
            NoteWindow(id: id).environment(store)
        }
    }
}

struct RootView: View {
    @SceneStorage("sel") private var sel: String? // per window
    var body: some View {
        NoteList(sel: $sel)
        .onOpenURL { sel = Route($0)?.noteID }
    }
}
```

Traps

- ! Saving on `.inactive` only, or in one window's `onChange` — use the App-level `.background`; per-window phases fire once per window.
- ! `@AppStorage` for a selection: two iPad windows overwrite each other — `@SceneStorage`.
- ! `.backgroundTask` with no submitted request / Info.plist id never runs; timing is the system's.
- ! `Window, Settings, MenuBarExtra` are not iOS — `#if os(macOS)`.
- ! **TN3187:** a UIKit app without the `UIScene` life cycle, built with the SDK after iOS 26, **won't launch**. SwiftUI App is scene-based already.

UIKit callback → SwiftUI

<code>didFinishLaunching</code>	<code>App.init</code> / adaptor
<code>sceneDidBecomeActive ...</code>	<code>onChange(of: scenePhase)</code>
<code>scene(_:openURLContexts:)</code>	<code>.onOpenURL</code>
<code>scene(_:continue:)</code>	<code>.onContinueUserActivity</code>
<code>stateRestorationActivity</code>	<code>@SceneStorage, path.codable</code>
<code>BGTaskScheduler.register</code>	<code>.backgroundTask(.appRefresh)</code>
<code>push token,</code>	the adaptor's delegate methods
<code>UNUserNotificationCenter delegate</code>	